

SEO PACKAGE

1. SEO Title

Python Scope, `*args`, and `**kwargs` Explained: Why Python Ignores Your Variable Change

2. SEO Subtitle

A beginner-friendly guide to local vs. global scope, the `global` keyword, and packing or unpacking any number of function arguments in Python.

3. Featured Quote

"Assigning to a name inside a function makes it local — even when a global variable shares that exact name."

4. Meta Description

Learn Python scope, the global keyword, and `args/**kwargs` with clear examples. Understand why reassigning a global silently fails and fix it fast.

5. URL Slug

`python-scope-args-kwargs-explained`

6. Keywords

Python scope, local vs global variables, Python global keyword, args and kwargs, Python *args*, Python **kwargs*, positional arguments, keyword arguments, variable scope Python, Python functions, argument unpacking, packing arguments, NameError Python, UnboundLocalError, Python for beginners, coding interview Python, LeetCode Python, Python data structures, software engineering, clean code Python

7. Tags

python python-basics args-kwars variable-scope global-keyword functions
coding-interview learn-python python-for-beginners clean-code algorithms
software-engineering programming python-tutorial

WHY DOES PYTHON IGNORE YOUR VARIABLE CHANGE?

Understanding Scope, `*args`, and `**kwargs` in Python

8. Introduction

You write a function, change a variable inside it, run your program — and the variable outside stays exactly the same. Or worse, Python throws an error complaining about a variable you *clearly* defined. If you have hit this wall, you have run into **scope**, one of the most misunderstood ideas in Python.

Scope is not an exotic topic. It quietly governs every function you will ever write. Interviewers love it because it separates developers who *memorized* syntax from those who actually understand how Python executes code. The same goes for `*args` and `**kwargs` — those "weird star names" that show up in real library code, decorators, API wrappers, and nearly every serious Python codebase.

In this lesson you will learn:

- What scope really means and who can see a variable
- Why reassigning a global variable inside a function silently fails
- When the `global` keyword actually helps (and when it burns you)
- How `*args` collects any number of positional arguments
- How `**kwargs` collects labeled keyword arguments
- The strict argument ordering Python enforces
- The reverse `*` / `**` trick for spreading collections back out

Master these and your functions stop feeling rigid. They become flexible tools that adapt to whatever you throw at them.

9. Problem Overview

There are really **two problems** bundled together here, and they share a root cause: understanding *where names live* in Python.

Problem one — Scope. When you create a variable, Python needs to know who is allowed to read it. A variable created inside a function belongs to that function. A variable created at the top level of your file belongs to everyone. Confusing the two leads to `NameError` (reading a name that does not exist yet) and `UnboundLocalError` (using a local name before it is assigned).

Problem two — Flexible arguments. Normally a function declares a fixed number of parameters. But real code often needs to accept *any* number of inputs — think of a `sum` function, a logging call, or a wrapper that forwards arguments to another function. Python solves this with `*args` (for unlabeled, positional values) and `**kwargs` (for labeled, keyword values).

Both problems are about the same skill: knowing what Python does with a name at the moment it sees it.

10. Example

Let's ground this with a mental model. Picture a **house**:

- Each **room** is a function.
- The **hallway** is your whole program (the global scope).

Something you leave in a bedroom stays in that bedroom. Something you leave in the hallway, everyone can reach.

```
def greet():  
    name = "Kai"      # 'name' lives only inside greet()  
    print(name)  
  
greet()              # prints: Kai  
print(name)          # NameError: name 'name' is not defined
```

Output explained: The first line prints `Kai` because we read `name` while `greet()` is still running. The second `print(name)` crashes — `name` was a secret the function kept to itself, and it vanished the moment `greet()` finished.

Now the hallway version:

```
count = 10                # global – lives in the hallway

def show():
    print(count)          # any function can READ a global

show()                    # prints: 10
```

Output explained: `show()` never defined `count`, so Python looks outward into the hallway, finds `count = 10`, and prints it. Reading a global from inside a function is perfectly fine.

11. Intuition

Here is the mental shortcut an experienced engineer uses: **Python decides a variable's scope based on whether you *assign* to it inside the function — not on whether a global with the same name exists.**

Reading is generous. If you only *read* a name, Python happily searches outward: local first, then the enclosing scope, then global. That is why `show()` could see `count`.

But *assignment* is possessive. The instant Python sees `score = ...` anywhere inside a function, it stamps `score` as **local to that function** — for the entire function, even before that line runs. This single rule explains almost every scope bug beginners hit.

Once that clicks, `*args` and `**kwargs` follow naturally. They are just Python's way of saying: "I do not know how many arguments are coming, so bundle the extras for me." A single `*` bundles loose *positional* values into a tuple. A double `**` bundles *labeled* values into a dictionary. The star does the packing; the name after it (`args` , `kwargs`) is only a convention.

12. Brute Force Solution

The naive approach to "accept many arguments": just add more parameters, or pass a list manually.

```
# Rigid: only ever handles exactly three numbers
def add_three(a, b, c):
    return a + b + c

# Slightly better, but the caller must build a list every time
def add_all(numbers):
    total = 0
    for n in numbers:
        total += n
```

```

    return total

add_all([1, 2, 3, 4])  # caller is forced to wrap values in []

```

Idea: Hard-code the number of parameters, or force callers to pack everything into a list themselves.

Algorithm: 1. Decide the maximum arguments you might need. 2. Declare that many parameters (or one list parameter). 3. Make every caller conform to that shape.

Advantages: - Dead simple to read. - No new syntax to learn.

Disadvantages: - `add_three` cannot handle four numbers without a rewrite. - `add_all` shifts the burden onto every caller — they must remember the brackets. - Neither scales to genuinely unknown input.

Time complexity: $O(n)$ to sum n values — the same as any approach. **Space complexity:** $O(n)$ for the list of values.

The complexity is fine. The *ergonomics* are not. This is where the optimal, Pythonic tools shine.

13. Optimal Solution

Reassigning a global — and the `global` keyword

Watch this common trap:

```

score = 0

def add():
    score = score + 1  # UnboundLocalError!
    print(score)

add()

```

Because `score` is **assigned** inside `add`, Python treats it as local *everywhere* in the function. So `score + 1` on the right tries to read a local `score` that has no value yet. The fix is to tell Python explicitly: reach into the hallway.

```

score = 0

def add():
    global score          # "use the outer score, don't make a new local"
    score = score + 1
    print(score)

```

```
add()                # prints: 1
print(score)         # prints: 1  - the global really changed
```

Use `global` **rarely**. A function that quietly rewrites globals is a function that surprises you at 3am.

`*args` — pack extra positional values into a tuple

```
def total(*nums):
    return sum(nums)    # nums is a tuple

total(1, 2)             # 3
total(1, 2, 3, 4, 5)   # 15
```

The `*` in front of `nums` is the magic. Whatever positional values you pass get bundled into a **tuple** named `nums`. Call it with two numbers or ten — the function just sums the bundle.

`**kwargs` — pack labeled values into a dictionary

```
def profile(**info):
    for key, value in info.items():
        print(f"{key} = {value}")

profile(name="Sam", age=9)
# name = Sam
# age = 9
```

The `**` packs every *named* argument into a **dictionary** called `info`. Keys are the labels; values are what you passed.

The pairing at a glance

Syntax	Catches	Bundled into
<code>*args</code>	loose positional values, in order	a tuple
<code>**kwargs</code>	labeled <code>name=value</code> pairs	a dictionary

Mixing them — order is strict

```
def demo(a, *args, **kwargs):
    print(a)        # 1
    print(args)     # (2, 3)
    print(kwargs)   # {'x': 9}

demo(1, 2, 3, x=9)
```

Python enforces the order: **plain parameters** → ***args** → ****kwargs** . Put them out of order and Python refuses to even run the file. Remember the phrase:

plain, then star, then star star.

The reverse trick — spreading collections back out

The star works both directions. In a *definition* it **collects**; at a *call* it **spreads**.

```
def add(a, b, c):
    return a + b + c

nums = [1, 2, 3]
add(*nums)                # spreads list → add(1, 2, 3) → 6

vals = {"a": 1, "b": 2, "c": 3}
add(**vals)               # spreads dict → add(a=1, b=2, c=3) → 6
```

nums** opens the list and hands each item over as a separate argument. *vals** matches dictionary keys to parameter names. It is like unpacking a suitcase straight into the drawers.

14. Python Code

```
def apply(func, *args, **kwargs):
    """Call `func` with any number of positional and keyword arguments.

    Demonstrates packing (*args, **kwargs) in the definition and the
    generic-wrapper pattern used throughout real Python codebases.
    """
    print(f"Positional args (tuple): {args}")
    print(f"Keyword args (dict):      {kwargs}")
    return func(*args, **kwargs)  # spread them back out into func

def add(a, b, c):
    return a + b + c

def describe(name, age):
    return f"{name} is {age}"

# Global-scope example done safely
counter = 0

def bump():
    global counter          # explicitly target the outer variable
```

```

counter += 1
return counter

if __name__ == "__main__":
    # --- self-check: prove the behavior instead of trusting it ---
    assert apply(add, 1, 2, 3) == 6
    assert apply(describe, name="Sam", age=9) == "Sam is 9"

    assert bump() == 1
    assert bump() == 2
    assert counter == 2          # the global really changed

    # spreading a list and a dict
    nums = [10, 20, 30]
    assert add(*nums) == 60
    assert add(**{"a": 1, "b": 2, "c": 3}) == 6

    print("All checks passed.")

```

15. Code Walkthrough

- **def apply(func, *args, **kwargs):** — a generic wrapper. `func` is a normal parameter; `*args` packs every extra positional value into a tuple; `**kwargs` packs every keyword value into a dict. This exact signature powers decorators and middleware.
- **return func(*args, **kwargs)** — the reverse trick. Here the stars *spread* the collected arguments back out, forwarding them untouched to `func`.
- **counter = 0** at module level — a global, readable everywhere.
- **global counter** inside `bump()` — without this line, `counter += 1` would raise `UnboundLocalError`, because the `+=` assignment would mark `counter` as local.
- **add(*nums)** — spreads the list `[10, 20, 30]` into three positional arguments.
- **add(**{...})** — spreads dictionary keys to matching parameter names.
- **The assert block** — every claim in this article is verified when you run the file. If any behavior breaks, the program fails loudly.

16. Dry Run

Let's trace `apply(add, 1, 2, 3)` step by step:

Step	What happens	State
1	<code>apply</code> is called with <code>func=add</code> , and <code>1, 2, 3</code> as extras	<code>args = (1, 2, 3)</code> , <code>kwargs = {}</code>
2	<code>*args</code> packs the three loose values into a tuple	<code>args = (1, 2, 3)</code>

Step	What happens	State
3	No keyword arguments were passed	<code>kwargs = {}</code>
4	<code>func(*args, **kwargs)</code> spreads back out	becomes <code>add(1, 2, 3)</code>
5	<code>add</code> returns <code>1 + 2 + 3</code>	<code>6</code>
6	<code>apply</code> returns that value	<code>6</code>

The value packs into a tuple on the way in, then spreads back into individual arguments on the way out.

17. Complexity Analysis

- **Time — $O(n)$:** Packing n values into a tuple, or n pairs into a dict, is linear. Spreading them back out is linear too. There is no hidden cost.
- **Space — $O(n)$:** `*args` allocates a tuple of size n ; `**kwargs` allocates a dict of size n . You are storing every argument once.

These bounds are correct because each argument is touched a constant number of times — packed once, optionally iterated once, spread once.

18. Alternative Solutions

Explicit list/dict parameters. Instead of `*args`, accept a single list; instead of `**kwargs`, accept a single dict.

```
def total(numbers):          # vs. def total(*nums)
    return sum(numbers)

total([1, 2, 3])            # caller must wrap in a list
```

Comparison:

Approach	Caller experience	Best for
<code>*args</code> / <code>**kwargs</code>	<code>total(1, 2, 3)</code> — clean	flexible public APIs, wrappers, decorators
explicit list/dict	<code>total([1, 2, 3])</code> — verbose	when the collection already exists as one object

Use `*args` / `**kwargs` when the *caller* has loose values. Use an explicit collection parameter when the data already lives in a list or dict.

19. Edge Cases

- **No arguments at all:** `total()` → `args` is an empty tuple `()`, `sum()` is `0`. No crash.
- **Only keyword arguments:** `apply(describe, name="Sam", age=9)` → `args` is empty, everything lands in `kwargs`.
- **Duplicate keys via spreading:** `add(**{"a": 1, "b": 2, "c": 3})` works; but passing both `a=1` positionally *and* in a spread dict raises `TypeError: got multiple values`.
- **Empty containers:** `add([])` passes nothing — you must still satisfy required parameters or you get a `TypeError`.
- **Reading a global before it exists:** referencing a name never defined anywhere raises `NameError`.

20. Common Mistakes

1. **Assuming assignment sees the global.** `score = score + 1` inside a function raises `UnboundLocalError` unless you declare `global score`.
2. **Reaching for global everywhere.** It makes code hard to trace. Prefer returning a value and reassigning: `score = add(score)`.
3. **Swapping `*` and `**`.** One star gives you a tuple; two give you a dict. Use the wrong one and your loop over "items" breaks.
4. **Wrong parameter order.** `def f(**kwargs, *args)` is a **SyntaxError**. It is always plain → `*args` → `**kwargs`.
5. **Forgetting to spread.** Passing `add(nums)` when `nums` is a list sends *one list* as the first argument, not three numbers. You need `add(*nums)`.
6. **Confusing "read" with "write" scope.** Reading a global works without `global`; only reassigning it requires the keyword.

21. Interview Questions

- What is the difference between `NameError` and `UnboundLocalError`?
- Why can a function *read* a global variable but not *reassign* it without the `global` keyword?
- What is the difference between `*args` in a function definition and `*args` at a call site?
- Explain the required order of `plain`, `*args`, and `**kwargs` parameters. What happens if you break it?
- How would you write a decorator that forwards all arguments to the wrapped function?
- What does `nonlocal` do, and how is it different from `global`?

22. Similar Problems

- **LeetCode 1929 — Concatenation of Array:** practices building and spreading collections, mirroring `*args` packing.
- **Function-composition and decorator problems:** rely directly on `*args, **kwargs` forwarding.
- **Two Sum (LeetCode 1):** while an algorithm problem, its clean solutions depend on tight, well-scoped local variables — the exact scope discipline taught here.
- **Design problems (e.g., LRU Cache, LeetCode 146):** stateful classes make you choose between instance state and globals; the scope reasoning here is the foundation.

These are related because every one of them rewards a clear mental model of *where a name lives* and *how arguments flow* into a function.

23. Key Takeaways

- **Scope answers one question:** who can read this variable? Locals stay in their function; globals live in the open.
- **Assignment makes a name local** for the whole function — the root cause of `UnboundLocalError`.
- **Use `global` sparingly.** Prefer passing values in and returning them out.
- `*args` packs loose positional values into a **tuple**; `**kwargs` packs labeled values into a **dictionary**.
- **Order is strict:** plain, then star, then star star.
- **The star works both ways:** collect in a definition, spread at a call.

24. Watch the Video

Reading builds understanding, but *watching* the concepts animate makes them stick. See the house-and-hallway model come to life, watch the `UnboundLocalError` happen live, and follow every `*args` / `**kwargs` example step by step.

 **Watch Lesson 12 here:** {LINK}

25. About the Series

This is **Lesson 12** of **Fun with Learning Technology** — a hands-on Python series that turns intimidating concepts into intuition-first lessons. Each episode takes one real stumbling block, explains *why* it works (not just *how*), and leaves you with production-quality code you can actually use. No jargon walls, no memorization — just clear teaching built for developers who want to genuinely understand the language.

26. Call To Action

If this cleared up scope or `*args` / `**kwargs` for you:

- 👍 **Like** the video on YouTube — it helps other learners find it.
- 🔔 **Subscribe** so you never miss a lesson in the series.
- 📌 **Subscribe to the Substack** for the written deep-dives and code.
- 💬 **Comment** with the scope bug that once cost *you* an afternoon.
- ➦ **Share** it with a developer who is still fighting `UnboundLocalError`.

Python may look hard. Subscribing is the easy part.

SOCIAL MEDIA

LinkedIn Post

Early in my career I lost an entire afternoon to a single Python bug. I had a global variable, I "changed" it inside a function, and Python flat-out ignored me — then threw an error about a variable I could clearly see was defined.

If you have written Python functions, you have probably hit this too. It comes down to one rule most tutorials skip:

The moment you assign to a name inside a function, Python treats that name as local — for the entire function, even if a global shares the exact same name.

That is why `score = score + 1` inside a function raises `UnboundLocalError`. The right-hand side tries to read a local that does not have a value yet. The fix is the `global` keyword — but honestly, you should reach for it rarely. A function that quietly rewrites globals is a function that surprises you at 3am.

The same "where does this name live?" thinking unlocks `*args` and `**kwargs`:

- A single `*` packs loose positional arguments into a tuple.
- A double `**` packs labeled arguments into a dictionary.
- Order is strict: plain parameters, then `*args`, then `**kwargs`.
- Both stars work in reverse too — at a call site they *spread* a list or dict back into individual arguments.

Understanding scope and argument packing is one of those quiet skills that separates developers who memorized syntax from those who understand how Python actually executes. It shows up in interviews, in every decorator, and in nearly every library you import.

I broke it all down in Lesson 12 of my Python series — house-and-hallway mental model included.

What Python bug once cost *you* an afternoon?

Python #SoftwareEngineering

#LearnToCode #CodingInterview

#Programming

Twitter/X Thread

- 1/ Ever changed a variable inside a Python function and watched Python completely ignore you? Or hit `UnboundLocalError` on a variable you *clearly* defined? Here's the one rule that explains it 🧠
- 2/ Mental model first. Picture a house. Each function is a room. The whole program is the hallway. Something left in a bedroom stays there. Something in the hallway, everyone can reach. That's scope.
- 3/ A variable created inside a function is LOCAL — it vanishes when the function ends. Try to print it outside and you get a `NameError`. It was a secret the function kept to itself.
- 4/ A variable at the top level is GLOBAL. Any function can READ it. That part is easy. The trap is trying to CHANGE it.
- 5/ The rule most tutorials skip: the moment Python sees you ASSIGN to a name inside a function, it marks that name local — for the whole function. Even if a global has the same name.
- 6/ So `score = score + 1` inside a function explodes. The right side tries to read a local `score` that has no value yet → `UnboundLocalError`. This bug has cost many devs an afternoon.
- 7/ The fix: the `global` keyword. It tells Python "use the hallway variable, don't make a new local." But use it rarely — functions that rewrite globals surprise you at 3am.

8/ Now the stars. `*args` packs any number of loose positional arguments into a `TUPLE`. `**kwargs` packs labeled arguments into a `DICTIONARY`. One star = order. Two stars = labels.

9/ Mixing them? Order is strict: plain params → `*args` → `**kwargs`. Break it and Python refuses to run the file. Remember: plain, then star, then star star.

10/ Bonus: the star works BOTH ways. In a definition it collects. At a call it spreads: `add(*my_list)` and `add(**my_dict)` unpack straight into arguments. Master this and your functions get wonderfully flexible. Full lesson + code 📌 {LINK}

Facebook Post

Quick Python question — have you ever changed a variable inside a function, run your code, and Python just... ignored the change? 🤔

You're not doing anything wrong. There's one rule tutorials love to skip: the second you assign to a name inside a function, Python treats it as *local* — even if a global has the same name. That's why `score = score + 1` can blow up with an error about a variable you can plainly see.

In Lesson 12 I break it all down with a simple house-and-hallway mental model, then tackle those "weird star names" — `*args` and `**kwargs` — that let your functions accept any number of arguments. Beginner-friendly, no jargon, real code you can run.

Come learn it with me 📌 {LINK}

Reddit Summary

Scope, `*args`, and `**kwargs` — the "why," not just the syntax

A lot of beginner confusion around Python functions comes down to one rule that rarely gets stated clearly:

Assigning to a name inside a function makes that name local for the *entire* function — even if a global variable shares the name.

That single rule explains both classic errors: - `NameError` — reading a name that doesn't exist in any reachable scope. - `UnboundLocalError` — e.g. `score = score + 1` inside a function, where the assignment marks `score` local, so the right-hand read fails before it's assigned.

Reading a global works fine without `global`. Only *reassigning* one needs it — and leaning on `global` tends to make code hard to trace, so returning values is usually cleaner.

On the argument side: - `*args` packs extra positional values into a **tuple**. - `**kwargs` packs labeled values into a **dict**. - Required order is plain → `*args` → `**kwargs` . - Both stars also *spread* at a call site: `f(*list)` , `f(**dict)` .

Wrote up a full explanation with runnable examples and a dry run if it's useful to anyone learning this. Happy to answer scope questions in the comments.

GITHUB README

```
# Python Scope, `*args`, and `**kwargs`
```

Lesson 12 of the **Fun with Learning Technology** Python series – understand where your variables live and how to accept any number of function arguments.

```
## Problem
```

Two closely related beginner stumbling blocks:

1. **Scope** – Why does changing a variable inside a function sometimes get ignored, and why does Python raise `UnboundLocalError` on a variable you clearly defined?
2. **Flexible arguments** – How do you write one function that accepts any number of positional or keyword arguments?

```
## Intuition
```

Think of a house. Each function is a **room**; the whole program is the **hallway** (global scope).

- A name created inside a function is **local** – it vanishes when the function returns.
- A name at the top level is **global** – any function can *read* it.
- **Key rule:** assigning to a name *anywhere* inside a function marks it **local for the entire function**, even if a global shares the name. That is the root cause of `UnboundLocalError`.

The stars follow the same "where does this live?" logic:

- `*args`` **packs** loose positional values into a **tuple**.
- `**kwargs`` **packs** labeled values into a **dictionary**.
- At a call site, the stars **spread** collections back into arguments.

```
## Approach
```

Tool	Definition side	Call side
<code>`*`</code>	collect positional → tuple	spread a list/tuple
<code>`**`</code>	collect keyword → dict	spread a dict

Parameter order is strict: `**plain` → `*args` → `**kwargs`. Use `global` sparingly; prefer returning values.

Python Solution

```
```python
def apply(func, *args, **kwargs):
 """Forward any number of arguments to func (the wrapper pattern)."""
 return func(*args, **kwargs)

def add(a, b, c):
 return a + b + c

counter = 0

def bump():
 global counter # target the outer variable explicitly
 counter += 1
 return counter

if __name__ == "__main__":
 assert apply(add, 1, 2, 3) == 6
 assert add(*[10, 20, 30]) == 60
 assert add(**{"a": 1, "b": 2, "c": 3}) == 6
 assert bump() == 1 and bump() == 2 and counter == 2
 print("All checks passed.")
```

## Complexity

---

- **Time:**  $O(n)$  to pack or spread  $n$  arguments.
- **Space:**  $O(n)$  for the tuple ( `*args` ) or dict ( `**kwargs` ).

## Video

---

 Watch Lesson 12: {LINK}

## Article

---

Full written deep-dive — mental models, dry run, edge cases, and common mistakes — available on the series Substack.

---



Part of the Fun with Learning Technology Python series. ``

## Quick Review

---

### Tell for \*args vs \*\*kwargs?

See many positional values needing one param → \*args (tuple). See many named key=value pairs → \*\*kwargs (dict).

### Cost of packing/unpacking N args?

O(N) time — Python builds a tuple (\*args) or dict (\*\*kwargs) by copying each argument once at call time.

### Space used by \*args/\*\*kwargs?

O(N) — a new tuple or dict holds all captured arguments; the collection lives for the call's duration.

### Common scope bug with globals?

Reassigning a global inside a function makes a new local instead; you must declare ``global x`` to rebind the outer variable.

### Correct parameter order in a signature?

positional, then \*args, then keyword-only, then \*\*kwargs. Wrong order (e.g. \*\*kwargs before \*args) is a SyntaxError.

### global vs the reassign trap?

Reading a global works freely; assigning shadows it locally. ``global`` forces assignment to hit the outer name — reads never needed it.

---

Python Lesson 12: Scope and \*args/\*\*kwargs (Local vs global scope, variable-length arguments, and unpacking) — free Python cheat sheet